Optimizing End-to-End Latency of Sporadic Cause-Effect Chains Using Priority Inheritance

Yue Tang[1], Xu Jiang[2*], Nan Guan[3], Songran Liu[1], Xiantong Luo[1], Wang Yi[1,4]

[1]Northeastern University, China [2]University of Electronic Science and Technology of China, China
[3]City University of Hong Kong, Hong Kong SAR [4]Uppsala University, Sweden

***Abstract*—Analysis and optimization of end-to-end latency in cause-effect chains is an important problem in real-time systems. Under task-level fixed-priority scheduling, the end-to-end latency largely relies on the relative priority of the tasks in the chain, so previous work has tried to improve the latency via priority assignment. However, the improvement of static priority assignment is limited due to the conflict between schedulability of individual tasks and end-to-end latency of the chain, i.e., a priority assignment leading to good end-to-end latency may make the task set unschedulable. This work proposes a novel method named Dynamic Priority Inheritance Protocol (DPI) to optimize the end-to-end latency of sporadic cause-effect chains. Under DPI, the propagation delay between two communicating jobs is independent of the task relative priority. So the optimization can work on any priority assignment, and no longer conflicts with task schedulability. Moreover, we propose DPI-B, a combination of DPI and a Buffer Manipulation Protocol, for cause-effect chains that also need to meet the determinism requirement. We conduct experiments with both automotive benchmarks and randomly generated workload. The results show the effectiveness of our method in comparison with the state-of-the-art.**

I. Introduction

Real-time systems in safety-critical domain are generally subject to end-to-end timing constraints. A cause-effect chain is a sequence of multi-rate real-time tasks with data dependency. The input stimulus is processed and propagated along the chain until an output is generated. A cause-effect chain usually implements a specific functionality (e.g., obstacle avoidance in automotive systems), and end-to-end latency of cause-effect chains describes how long it takes to complete the desired functionality. It is essential to perform end-to-end latency analysis and optimization to ensure the timing constraints are satisfied in the worst case.

End-to-end latency analysis of cause-effect chains is an important topic that attracts considerable attentions from both the academic research community and industry in recent years [1]–[5]. However, most existing work focuses on cause-effect chains composed of *periodic* tasks, where the scheduling behavior repeats after a certain length of time interval [6]. In this work, we study the more challenging case of sporadic tasks, where the release time pattern may never repeat and it is impossible to find a repeating time interval as in the periodic task setting. Dürr et.al [7] present the state-of-the-art results for analyzing sporadic tasks with implicit communication under fixed-priority preemptive scheduling, where the end-to-end latency is analyzed by bounding the length of time interval for the data to propagate from one task to the next. As pointed out in [7], the time interval between two successive tasks engaged in the data propagation largely relies on their relative priorities, where a task pair with low-to-high priority may result in a larger data propagation time in the worst-case. Consequently, an intuitive way to optimize the end-to-end delay is to assign tasks in decreasing priorities [7]. However, this may increase the response time of some tasks and even cause the system unschedulable. For other static priority assignment to make the system schedulable, low-to-high task pairs may unavoidably exist, and thus limiting the improvement of end-to-end latency.

In this work, we present a novel method named Dynamic Priority Inheritance protocol (DPI) to optimize the end-to-end latency in sporadic cause-effect chains. Under DPI, a job may dynamically inherit the priority from another job engaged in the data propagation, so that the propagation delay between two communicating jobs is independent of the task priorities. As a result, the protocol can effectively work upon any proper priority assignment, and schedulability becomes the unique consideration for priority assignment. We also prove that the upper bounds of end-to-end latency under DPI outperform those derived in [7].

Moreover, we propose DPI-B, which is a combination of the proposed DPI and a Buffer Manipulation Protocol, to implement data-flow determinism. Determinism is critical for model-based system design, allowing abstraction from implementation concerns and making designs platform-independent. Determinism is always implemented with heavy cost in terms of end-to-end latency, i.e., under LET [8], [9] the logical finish time of a job is fixed as its deadline, which is much larger than its actual finish time. However, benefiting from the run-time priority adjustment under DPI, our approach achieves the desired determinism with insignificant end-to-end latency increase.

Experiments with both automotive benchmarks and randomly generated workload are conducted to evaluate the proposed techniques. Experimental results show that the proposed DPI and DPI-B generate smaller end-to-end latency compared with the state-of-the-art.

*Xu Jiang is the corresponding author.

2023 IEEE Real-Time Systems Symposium (RTSS) | 979-8-3503-2857-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/RTSS59052.2023.00042

2576-3172/23/$31.00 ©2023 IEEE
DOI 10.1109/RTSS59052.2023.00042

II. System Model

A. Cause-Effect Chains

We consider a system consisting of a set of tasks executing on several Electronic Control Unit (ECU) characterized as processors, where each task is statically mapped onto a ECU. Tasks mapped onto the same ECU form a set of independent cause-effect chains (called chains for short) with respect to their inter-data dependency. A chain in a ECU, denoted by $\mathcal{C} = \{\tau_1, \tau_2, ..., \tau_j, ..., \tau_{|\mathcal{C}|}\}$, consists of an ordered sequence of tasks, where τ_j is the j^{th} task in $\mathcal{C}$, and $|\mathcal{C}|$ is the number of tasks in $\mathcal{C}$. The Worst-Case Execution Time (WCET) e_j of each task $\tau_j \in \mathcal{C}$ is known in advance. We call τ_j the predecessor of τ_{j+1}, and τ_{j+1} the successor of τ_j. There is a buffer of size 1 in between each pair of consecutive tasks τ_j and τ_{j+1} in $\mathcal{C}$, named the output buffer of τ_j and the input buffer of τ_{j+1}. In the following, we focus on chains in an arbitrary ECU, and later in Section IV.D, we discuss the inter-connections among different ECUs.

At run-time, each task sporadically releases an infinite number of jobs. We use $T_j^{\min}$ and $T_j^{\max}$ to denote the minimum and maximum inter-release separation between two consecutive jobs of τ_j, respectively. The k^{th} job released by τ_j is denoted by J_j^k. All jobs released by tasks on the same ECU are scheduled by a Fixed-Priority Preemptive (FPP) scheduler. Each task τ_j is assigned with a unique priority $\pi(\tau_j)$, and each job inherits the priority of the task releasing it. In particular, τ_j has higher priority than τ_k if $\pi(\tau_j) < \pi(\tau_k)$ holds. The *release time*, *start time* and *finish time* of J_j^k are denoted by $r(J_j^k)$, $s(J_j^k)$ and $f(J_j^k)$, respectively. A job is *pending* at t if it has been released and not finished at t. The response time of J_j^k is denoted by $R(J_j^k) = f(J_j^k) - r(J_j^k)$, and the response time of task τ_j is denoted by $\mathcal{R}(\tau_j)$, which is the maximal response time among all its jobs. Tasks are assumed to have implicit deadlines, i.e., $T_j^{\min}$ is also the relative deadline of τ_j, and τ_j is *schedulable* if $\mathcal{R}(\tau_j) \leq T_j^{\min}$. The problem of bounding $\mathcal{R}(\tau_j)$ has been extensively studied in literature [10]. In this paper, we do not focus on the schedulability of the system, and simply assume that each task is schedulable under FPP.

The communication behaviors among jobs comply with the implicit semantics [1], which is commonly implemented in automotive systems, e.g, AUTOSAR. More specifically, a job J_j^k reads the data from its input buffer at the beginning of its execution, i.e., $s(J_j^k)$, and produces an output data into its output buffer at the end of its execution, i.e., $f(J_j^k)$. In this case, we say the output data is *caused* by the input data. The old data in a buffer is overwritten when a new data arrives. In particular, jobs released by the first task in a chain read data from some external source, such as the sensors.

We assume time is discrete. For any time $t > 0$, the notations t^- and t^+ are used to denote the time $t-\epsilon$ and $t+\epsilon$, respectively, where ϵ is an arbitrarily small positive number.

B. End-to-End Latency and Job Chains

In cause-effect chains, two types of end-to-end latency semantics are of most concern: the maximum *reaction time* and the maximum *data age*, which is also known as the first-to-first delay and last-to-last delay, respectively [11]:

- The maximum reaction time is the maximal delay that system needs to react to external events at the input that can arrive asynchronously at any time.
- The maximum data age is the maximal delay between the last input (that is not overwritten) until the last output (even in case of duplicates).

Dürr et.al [7] introduced alternative definitions of the maximum reaction time and maximum data age referring to the lengths of so-called *immediate forward/backward job chains*. An immediate forward job chain $\overrightarrow{c}$ on $\mathcal{C}$, denoted by $\overrightarrow{c} \in \mathcal{C}$, is defined from the perspective of an input of $\mathcal{C}$, whereas an immediate backward job chain $\overleftarrow{c} \in \mathcal{C}$ is defined from the perspective of an output produced by the last task in $\mathcal{C}$.

Definition 1 (Immediate Forward Job Chain) *Let* $\overrightarrow{c} = \{\overrightarrow{c}_1, \overrightarrow{c}_2, ..., \overrightarrow{c}_j, ...\overrightarrow{c}_{|\mathcal{C}|}\}$ *denote an immediate forward job chain on* $\mathcal{C}$*, where* $\overrightarrow{c}_j$ *is the* j^{th} *job in* $\overrightarrow{c}$ *and is released by* τ_j*. Then it satisfies:* $\forall j \in [1, |\mathcal{C}| - 1]$, $\overrightarrow{c}_{j+1}$ *is the job released by* τ_{j+1} *with the earliest start time after* $\overrightarrow{c}_j$ *finishes. The length of* $\overrightarrow{c}$ *is denoted by* $L(\overrightarrow{c}) = f(\overrightarrow{c}_{|\mathcal{C}|}) - r(\overrightarrow{c}_1)$.

Definition 2 (Immediate Backward Job Chain) *Let* $\overleftarrow{c} = \{\overleftarrow{c}_1, \overleftarrow{c}_2, ..., \overleftarrow{c}_j, ..., \overleftarrow{c}_{|\mathcal{C}|}\}$ *denote an immediate backward job chain on* $\mathcal{C}$*, where* $\overleftarrow{c}_j$ *is the* j^{th} *job in* $\overleftarrow{c}$ *and is released by* τ_j*. Then it satisfies:* $\forall j \in [1, |\mathcal{C}|-1]$, $\overleftarrow{c}_j$ *is the last job released by* τ_j *that finishes before* $\overleftarrow{c}_{j+1}$ *starts. The length of* $\overleftarrow{c}$ *is denoted by* $L(\overleftarrow{c}) = f(\overleftarrow{c}_{|\mathcal{C}|}) - r(\overleftarrow{c}_1)$.

The maximum reaction time and maximum data age are then defined with the length of immediate forward/backward job chains, respectively [7].

Definition 3 (Maximum Reaction Time) *The reaction time of cause-effect chain* $\mathcal{C}$ *is defined as*

$$RT(\mathcal{C}) = \max_{\forall \overrightarrow{c} \in \mathcal{C}} L(\overrightarrow{c}) + T_1^{max}$$

where $\overrightarrow{c}$ *is an arbitrary immediate forward job chain corresponding to* $\mathcal{C}$.

Definition 4 (Maximum Data Age) *The data age of cause-effect chain* $\mathcal{C}$ *is defined as*

$$DA(\mathcal{C}) = \max_{\forall \overleftarrow{c} \in \mathcal{C}} L(\overleftarrow{c})$$

where $\overleftarrow{c}$ *is an arbitrary immediate backward job chain corresponding to* $\mathcal{C}$.

It is necessary to emphasize that, in Definition 3, the length of an immediate forward job chain in the worst-case is extended with the maximum inter-release time of the first task. This is necessary since an external event can arrive immediately after a job of the first task starts its execution.

An example. Figure 1 shows the execution sequence of jobs released by tasks in a cause-effect chain $\mathcal{C} = \{\tau_1, \tau_2, \tau_3\}$. τ_1, τ_2 and τ_3 are all sporadically released with the minimum and maximum inter-release times of 5 and 10, respectively. The up and down black arrows denote the release and finish

time of each job. τ_1 has the highest priority and τ_2 the lowest priority. The WCETs of these three tasks are: $e(\tau_1) = 1$, $e(\tau_2) = 2$ and $e(\tau_3) = 2$. The colors of rectangles specify the priority each job executes with, and the white rectangle means that the job is not executing during the time interval. J_2^2 is the job with the earliest start time after J_1^2 finishes, and thus is the first job to read the data written by J_1^2. Similarly, J_3^3 is the job with the earliest start time after J_2^2 finishes. So by Definition 1, the immediate forward job chain starting from J_1^2 is $\{J_1^2, J_2^2, J_3^3\}$ (denoted by the red arrow). J_2^3 is the latest job which finishes before the start of J_3^4, and thus the output of J_3^4 is caused by the data produced by J_2^3. Similarly, the output of J_2^3 is caused by the data produced by J_1^4. So by Definition 2, the immediate backward job chain ending at J_3^4 is $\{J_1^4, J_2^3, J_3^4\}$ (denoted by the red dashed arrow).

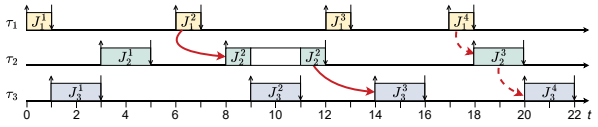


Fig. 1. Illustration of definitions.

III. Motivation

As shown in equations (1) and (2), which are directly deduced from the definitions of the maximum reaction time and data age, $RT(\mathcal{C})$ and $DA(\mathcal{C})$ are actually decided by the difference of release times $r(\vec{c}_j) - r(\vec{c}_{j-1})$ (respectively, $r(\overleftarrow{c}_j) - r(\overleftarrow{c}_{j-1})$) between each pair of consecutive jobs in $\vec{c}$ (respectively, $\overleftarrow{c}$) in the worst-case scenario, which, in fact, largely depends on their relative priorities under the precondition that all tasks in the chain are **schedulable**.

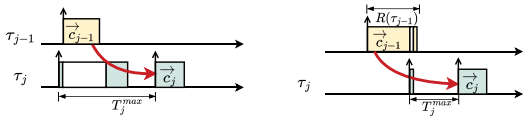
For example, in Figure 2-(a), the difference of release times between $\vec{c}_{j-1}$ and $\vec{c}_j$ is upper-bounded by T_j^{max}, since any job released by τ_j no earlier than $r(\vec{c}_{j-1})$ must start execution after $\vec{c}_{j-1}$ finishes. Inversely, in Figure 2-(b), the difference of release times between $\vec{c}_{j-1}$ and $\vec{c}_j$ is upper-bounded by $\mathcal{R}(\tau_{j-1}) + T_j^{max}$, when $\vec{c}_{j-1}$ is preempted by a job of τ_j released a little earlier than the finish time of $\vec{c}_{j-1}$. As a result a later job $\vec{c}_j$ of τ_j is the first one which starts after $\vec{c}_{j-1}$ finishes. Similar results have been reported by Dürr et.al in Theorem 5.4 and 5.10 of [7]. Following similar perspective, some previous work proposed priority assignment strategy to achieve better end-to-end latency. For example, [7] proposed that setting the priorities of all tasks in a chain to be decreasing greatly shortens the end-to-end latency.

$$\begin{aligned} RT(\mathcal{C}) &= \max_{\vec{c} \in \mathcal{C}} L(\vec{c}) + T_1^{max} \quad \textit{(by Definition 3)} \\ &= \max_{\vec{c} \in \mathcal{C}} \{f(\vec{c}_{|\mathcal{C}|}) - r(\vec{c}_1)\} + T_1^{max} \quad \textit{(by Definition 1)} \\ &= \max_{\vec{c} \in \mathcal{C}} \{\sum_{j=2}^{|\mathcal{C}|} (r(\vec{c}_j) - r(\vec{c}_{j-1})) + R(\vec{c}_{|\mathcal{C}|})\} + T_1^{max} \end{aligned} \tag{1}$$

$$\begin{aligned} DA(\mathcal{C}) &= \max_{\overleftarrow{c} \in \mathcal{C}} L(\overleftarrow{c}) \quad \textit{(by Definition 4)} \\ &= \max_{\overleftarrow{c} \in \mathcal{C}} \{f(\overleftarrow{c}_{|\mathcal{C}|}) - r(\overleftarrow{c}_1)\} \quad \textit{(by Definition 2)} \\ &= \max_{\overleftarrow{c} \in \mathcal{C}} \{\sum_{j=2}^{|\mathcal{C}|} (r(\overleftarrow{c}_j) - r(\overleftarrow{c}_{j-1})) + R(\overleftarrow{c}_{|\mathcal{C}|})\} \end{aligned} \tag{2}$$

However, to make the task set schedulable and to shorten the end-to-end latency may be two conflicted targets when finding a static priority assignment. On one hand, setting the priorities of all tasks in a chain to be decreasing as in [7] may increase the response time of some tasks and even make the task set unschedulable, leading to an unpredicatable end-to-end latency. On the other hand, a priority assignment resulting in a schedulable task set may unavoidably include communicating tasks with a lower-priority writer task and a higher-priority reader task, which as shown above will lengthen the end-to-end latency. Therefore, a static priority assignment may not always be a good choice.

Inspired by the above observations, in this paper, we propose a run-time protocol to optimize end-to-end latency in sporadic cause-effect chains. The key idea is to dynamically adapt the priority of the reader job between two consecutive jobs in an immediate forward/backward job chain by using priority inheritance. In particular, our protocol can adapt to any priority assignment and optimize the end-to-end delay.



(a) $\pi(\tau_{j-1}) < \pi(\tau_j)$ (b) $\pi(\tau_{j-1}) > \pi(\tau_j)$

Fig. 2. Worst-case scenarios under different cases.

IV. The Dynamic Priority Inheritance Protocol

In this section, we introduce the protocol, named Dynamic Priority Inheritance protocol (DPI), which dynamically specifies priorities to jobs at run-time.

A. Dynamic Priority Inheritance Protocol

The intuition of DPI is to lower the run-time priority of some job so that it can read the data produced by a latest released job of its predecessor task. In particular, each job J_j^k is scheduled with an *effective priority* $\pi'(J_j^k)$, which is decided when J_j^k is released and may be different from $\pi(\tau_j)$. To avoid ambiguity, we use the following definition to distinguish schedules resulted by the original FPP and DPI.

Definition 5 *A schedule S under FPP is an execution sequence of all jobs on the ECU resulted by FPP. The correspondence of S under DPI, denoted by S', is defined as the execution sequence resulted by DPI where the release pattern and execution time of all jobs are exactly the same as that in S.*

Accordingly, we distinguish notations under S and S' by prime $'$, e.g., $f(J_j^k)$ and $f'(J_j^k)$ denote the finish time of J_j^k

under S and S', respectively. In the following, we introduce the scheduling under DPI, considering an arbitrary schedule S'. Then we present some important properties under DPI.

Definition 6 (Basal Priority) *The basal priority of tasks τ_i and τ_j at t, denoted by $\Pi_{i,j}(t)$, is defined as the lowest effective priority among all jobs of both τ_i and τ_j that are pending at t. If no jobs of τ_i and τ_j are pending at t, $\Pi_{i,j}(t) = 0$.*

At run-time, jobs are scheduled under DPI according to the following rules.

Rule 1. A job J_j^k continues to execute until it finishes even if it misses its deadline.[1]

Rule 2. If $j = 1$, then $\pi'(J_j^k) = \pi(\tau_j)$. Otherwise, let J_j^k be released at t. Then $\pi'(J_j^k) = max(\pi(\tau_j), \Pi_{j,j-1}(t))$, i.e., J_j^k inherits the lower priority between $\pi(\tau_j)$ and the lowest priority among jobs of both τ_j and τ_{j-1} that are pending at t. Recall that a smaller number implies higher priority.

Rule 3. When more than two pending jobs have the same priority, the one with earlier release time executes first.

An example. Figure 3 shows the execution sequence of a chain $\mathcal{C} = \{\tau_1, \tau_2, \tau_3\}$ (The level-i busy period marked in the figure will be used later in Section IV.C.). The priorities of tasks are set to $\pi(\tau_1) = 3$, $\pi(\tau_2) = 1$, and $\pi(\tau_3) = 2$ with their respective periods $T_1^{min} = 20$, $T_2^{min} = 5$ and $T_3^{min} = 6$. Different colors are used to differentiate the priorities of each job, and the white box indicates that the job is not executing.

By Rule 2, the effective priority of J_1^1 is equal to $\pi(\tau_1)$, i.e., $\pi'(J_1^1) = \pi(\tau_1) = 3$. At time 1, the basal priority of τ_1 and τ_2 is $\Pi_{2,1}(1) = \pi'(J_1^1) = 3$ (Definition 6), and $\pi'(J_2^1) = max(\pi(\tau_2), \Pi_{2,1}(1)) = 3$ (Rule 2). At time 6, J_2^1 misses its deadline, and it continues to execute to finish (Rule 1). J_2^2 has the same effective priority as J_2^1 and does not start until J_2^1 finishes (Rule 3). At time 7, the basal priority of τ_2 and τ_3 is $\Pi_{3,2}(7) = \pi'(J_2^2) = 3$ (Definition 6), and $\pi'(J_3^1) = max(\pi(\tau_3), \Pi_{3,2}(7)) = 3$ (Rule 2). J_3^1 and J_2^2 have the same effective priority, and J_2^2 executes first since it has an earlier release time (Rule 3). At time 11, J_2^3 is released. Since there are no pending jobs released by τ_1 and τ_2, $\Pi_{2,1}(11) = 0$, and $\pi'(J_2^3) = max(\pi(\tau_2), \Pi_{2,1}(11)) = 1$.

In the next, we introduce some properties under DPI.

Lemma 1 $\pi'(J_j^k) \geq \pi(J_j^k)$.

Proof: Directly derived by Rule 2. □

Lemma 2 *J_j^p cannot start to execute until $f'(J_j^q)$ if $p > q$.*

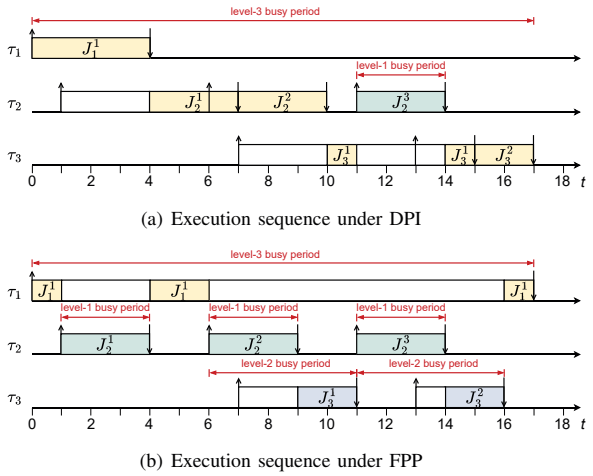
Proof: Proved by combining Rule 2 and Rule 3. □

Lemma 3 *Let J_{j-1}^p be the latest job released by τ_{j-1} before J_j^k, then J_j^k must start execution after $f'(J_{j-1}^p)$.*

Proof: We distinguish two cases.

- $f'(J_{j-1}^p) \leq r(J_j^k)$. Then the lemma obviously holds.

[1]Rule 1 is an alternative design choice. We can also choose to directly discard the jobs that miss their deadlines and get similar results that derived in the rest of paper.



(a) Execution sequence under DPI

(b) Execution sequence under FPP

Fig. 3. Comparison of execution sequence under DPI and FPP.

- $f'(J_{j-1}^p) > r(J_j^k)$. Then J_{j-1}^p is pending at $r(J_j^k)$. By Definition 6 and Rule 2, $\pi'(J_j^k) \geq \pi(J_{j-1}^p)$. Then J_j^k can not start before J_{j-1}^p finishes. Then the lemma is proved.

Combining the two cases, the lemma is proved. □

Lemma 4 *J_j^p cannot start to execute until $f'(J_{j-1}^q)$ if J_j^p is released later than J_{j-1}^q.*

Proof: Proved by combining Lemma 2 and 3. □

Lemma 5 *If it satisfies for a job J_j^k that $\pi'(J_j^k) = \pi(\tau_i) > \pi(\tau_j)$ where $\tau_i \neq \tau_j$, then $j > i$ must hold.*

Proof: According to Rule 2, the effective priority of a job can only be inherited from either the task releasing it or jobs released by its predecessor task in the chain. By induction, the lemma is true. □

B. End-to-End Latency

In the following, we present analysis for the end-to-end latency of a chain scheduled under DPI. We first upper-bound the difference of release time between two consecutive jobs in an immediate forward/backward job chain, respectively, and then derive the maximum reaction time and maximum data age.

Lemma 6 *Let $\overrightarrow{c}_{j-1}$ and $\overrightarrow{c}_j$ be two consecutive jobs in an arbitrary immediate forward job chain $\overrightarrow{c}$. Then*

$$r(\overrightarrow{c}_j) - r(\overrightarrow{c}_{j-1}) \leq T_j^{max}$$

Proof: We distinguish two cases.

- $\overrightarrow{c}_j$ is released no later than $\overrightarrow{c}_{j-1}$. Then $r(\overrightarrow{c}_j) - r(\overrightarrow{c}_{j-1}) \leq 0$, and the lemma holds.
- $\overrightarrow{c}_j$ is released after $\overrightarrow{c}_{j-1}$. We prove by contradiction. Suppose that $r(\overrightarrow{c}_j) - r(\overrightarrow{c}_{j-1}) > T_j^{max}$. Then there must be another job J_j^p of τ_j released after $\overrightarrow{c}_{j-1}$, and $r(J_j^p) < r(\overrightarrow{c}_j)$ holds. According to Lemma 4, we know J_j^p must start execution after $f'(\overrightarrow{c}_{j-1})$. Moreover, according to Lemma 2, the start time of J_j^p must be earlier

than the start time of $\overrightarrow{c}_j$. Reaching a contradiction with the definition that $\overrightarrow{c}_j$ is the job released by τ_j with the earliest start time after the finish time of $\overrightarrow{c}_{j-1}$. Therefore, $r(\overrightarrow{c}_j) - r(\overrightarrow{c}_{j-1}) \leq T_j^{max}$ holds, and the lemma is proved.

Combining the two cases, the lemma is proved. □

Lemma 7 *Let $\overleftarrow{c}_{j-1}$ and $\overleftarrow{c}_j$ be two consecutive jobs in an arbitrary immediate backward job chain $\overleftarrow{c}$. Then*

$$r(\overleftarrow{c}_j) - r(\overleftarrow{c}_{j-1}) \leq T_{j-1}^{max}$$

Proof: We distinguish two cases.

- $\overleftarrow{c}_j$ is released no later than $\overleftarrow{c}_{j-1}$. Then $r(\overleftarrow{c}_j) - r(\overleftarrow{c}_{j-1}) \leq 0$, and the lemma holds.
- $\overleftarrow{c}_j$ is released after $\overleftarrow{c}_{j-1}$. We prove by contradiction. Suppose that $r(\overleftarrow{c}_j) - r(\overleftarrow{c}_{j-1}) > T_{j-1}^{max}$. Then there must be a job J_{j-1}^p of τ_{j-1} released before $\overleftarrow{c}_j$, and $r(J_{j-1}^p) > r(\overleftarrow{c}_{j-1})$ holds. According to Lemma 4, we know $\overleftarrow{c}_j$ must start execution after $f'(J_{j-1}^p)$. Moreover, according to Lemma 2, J_{j-1}^p must be finished later than the finish time of $\overleftarrow{c}_{j-1}$. Reaching a contradiction with the definition that $\overleftarrow{c}_{j-1}$ is the last job released by τ_{j-1} that finishes before $\overleftarrow{c}_j$ starts. Therefore, $r(\overleftarrow{c}_j) - r(\overleftarrow{c}_{j-1}) \leq T_{j-1}^{max}$ holds.

Combining the two cases, the lemma is proved. □

The maximum reaction time and maximum data age under DPI are derived by Theorem 1 and 2.

Theorem 1 *The reaction time of $\mathcal{C}$ is upper-bounded by*

$$RT(\mathcal{C}) \leq T_1^{max} + \sum_{j=2}^{|\mathcal{C}|} T_j^{max} + \max_{\forall u}\{R'(J_{|\mathcal{C}|}^u)\} \tag{3}$$

Proof: According to Definition 3 and equation (1):

$$RT(\mathcal{C}) = T_1^{max} + \max_{\forall \overrightarrow{c} \in \mathcal{C}} \left(\sum_{j=2}^{|\mathcal{C}|} (r(\overrightarrow{c}_j) - r(\overrightarrow{c}_{j-1})) + R'(\overrightarrow{c}_{|\mathcal{C}|}) \right)$$

$$\leq T_1^{max} + \sum_{j=2}^{|\mathcal{C}|} T_j^{max} + \max_{\forall u}\{R'(J_{|\mathcal{C}|}^u)\} \quad (\textit{by Lemma 6})$$

□

Theorem 2 *The data age of $\mathcal{C}$ is upper-bounded by*

$$DA(\mathcal{C}) \leq \sum_{j=1}^{|\mathcal{C}|-1} T_j^{max} + \max_{\forall u}\{R'(J_{|\mathcal{C}|}^u)\} \tag{4}$$

Proof: By Definition 4 and equation (2):

$$DA(\mathcal{C}) = \max_{\forall \overleftarrow{c} \in \mathcal{C}} \left(\sum_{j=2}^{|\mathcal{C}|} r(\overleftarrow{c}_j) - r(\overleftarrow{c}_{j-1}) + R'(\overleftarrow{c}_{|\mathcal{C}|}) \right)$$

$$\leq \sum_{j=1}^{|\mathcal{C}|-1} T_j^{max} + \max_{\forall u}\{R'(J_{|\mathcal{C}|}^u)\} \quad (\textit{by Lemma 7})$$

□

From Theorem 1 and 2, the problems of bounding the maximum reaction time and data age degrade to the problem of bounding $\max_{\forall u}\{R'(J_{|\mathcal{C}|}^u)\}$. It must be noticed that, since jobs are allowed to miss their deadlines, $R'(J_{|\mathcal{C}|}^u)$ is not necessarily bounded by $T_{|\mathcal{C}|}^{min}$ even under the assumption that the system is schedulable under original FP scheduler. In the next, we focus on bounding $\max_{\forall u}\{R'(J_{|\mathcal{C}|}^u)\}$, i.e., $\mathcal{R}'(\tau_{|\mathcal{C}|})$.

C. Bounding $\max_{\forall u}\{R'(J_{|\mathcal{C}|}^u)\}$

While the upper-bound of the difference of release time between two communicating jobs is only decided by the release pattern of tasks in the analyzed chain, the response time of an arbitrary job is decided by the execution of all tasks on the ECU. In this section we consider that DPI has been applied on multiple chains in the ECU, and calculate the upper-bound of response time of an arbitrary job in the analyzed chain. Before going deep, we first introduce a concept named level-i busy period [12].

Definition 7 (Level-i Busy Period) *A time interval $[a, b)$ is called a level-i busy period if:*

- *the processor is busy executing jobs with effective priorities higher than or equal to i at any time point in $[a, b)$.*
- *at least one job with effective priority i has been scheduled in $[a, b)$.*
- *no jobs with priority higher than or equal to i are pending at both time a^- and b^-.*

Let J_j^k be an arbitrary job released by τ_j (in both S and S'), where $\pi(\tau_j) = i$. In S, no jobs with priority higher than or equal to $\pi(\tau_j)$ are pending at $f(J_j^k)$ (otherwise, J_j^k cannot finish at $f(J_j^k)$). Let t be the latest time point before $r(J_j^k)$ when no jobs with priority higher than or equal to $\pi(\tau_j)$ are pending. Then by definition, $[t, f(J_j^k))$ is a level-i busy period, and we call $[t, f(J_j^k))$ the busy period of J_j^k.

Let $work_i(\alpha, \beta)$ (respectively, $work_i'(\alpha, \beta)$) denote the total amount workload of jobs with release times falling in $[\alpha, \beta)$ and priorities higher than or equal to i under S (respectively, S'). Then it satisfies that:

Lemma 8 $work_i(\alpha, \beta) \geq work_i'(\alpha, \beta)$.

Proof: Since all jobs have the same release pattern and execution time under S and S', then it is sufficient to prove the lemma by proving that $\pi'(J_m^n) \geq \pi(\tau_m)$ satisfies for each job J_m^n in S', where τ_m is an arbitrary task on the ECU, which is true according to Lemma 1. □

For simplicity, we use $\mathcal{J}_i(t)$ (respectively, $\mathcal{J}_i'(t)$) to denote the set of all jobs with priority higher than or equal to i that are pending at t in S (respectively, S').

Lemma 9 *Let $[\alpha, \beta)$ be the busy period of J_j^k under S, where $\pi(\tau_j) = i$. Then $\mathcal{J}_i'(\alpha^-) = \emptyset$ and $\mathcal{J}_i'(\beta^-) = \emptyset$ hold under S'.*

Proof: By definition, we know $\mathcal{J}_i(\alpha^-) = \emptyset$ holds under S. Since $work_i(0, \alpha) \geq work_i'(0, \alpha)$ (by Lemma 8), then $\mathcal{J}_i'(\alpha^-) = \emptyset$ must hold under S'. Moreover, since $\mathcal{J}_i(\beta^-) = \emptyset$

holds under S and $work_i(\alpha,\beta) \geq work'_i(\alpha,\beta)$, we know $\mathcal{J}'_i(\beta^-) = \emptyset$ must hold under S', and the lemma is proved. □

In the following, we derive the upper-bounds of $R'(J_j^k)$ in different cases: $\pi'(J_j^k) = \pi(\tau_j)$ and $\pi'(J_j^k) > \pi(\tau_j)$.

Lemma 10 *If $\pi'(J_j^k) = \pi(\tau_j)$, then $R'(J_j^k) \leq \mathcal{R}(\tau_j)$ holds.*

Proof: According to Lemma 9, no jobs with priority higher than or equal to $\pi(\tau_j)$ are pending at $f(J_j^k)$ (i.e., the finish time of J_j^k in S) in S', i.e., $\mathcal{J}'_{\pi(\tau_j)}(f(J_j^k)^-) = \emptyset$. Then, we know $f'(J_j^k) \leq f(J_j^k)$. Therefore $R'(J_j^k) \leq R(J_j^k) \leq \mathcal{R}(\tau_j)$. The lemma is proved. □

Lemma 11 *Let $\mathcal{L}$ denote the set of tasks in $\mathcal{C}$ where it satisfies that $\pi(\tau_i) > \pi(\tau_{i+1})$ for each task $\tau_i \in \mathcal{L}$. If $\pi'(J_j^k) > \pi(\tau_j)$, then it satisfies that $R'(J_j^k) \leq \max_{\forall \tau_i \in \mathcal{L}}\{\mathcal{R}(\tau_i)\}$.*

Proof: Let $\pi(\tau_x) = \pi'(J_j^k) > \pi(\tau_j)$. According to Rule 2 of DPI, there must be at least one job with priority $\pi(\tau_x)$ pending at $r(J_j^k)$ in S'. Let $t < r(J_j^k)$ denote the latest time point before $r(J_j^k)$ when no jobs with priority higher than or equal to $\pi(\tau_x)$ are pending in S'.

We first prove that at least one job of τ_x must be released in $[t, r(J_j^k))$. Let J be the job with the earliest release time among all jobs with effective priority equal to $\pi(\tau_x)$ that are released in $[t, r(J_j^k))$ in S', and r denote the release time of J. Suppose that J is not released by τ_x. Then according to Rule 2 of DPI, there must be a job with effective priority $\pi(\tau_x)$ pending at r, i.e., released before r. Reaching a contradiction with either the definition of t (if the job is released earlier than t) or the definition of J (if the job is released no earlier than t). Therefore, J must be released by τ_x, denoted by J_x^p.

Then we prove that the finish time of J_x^p under S must be no earlier than $f'(J_j^k)$. According to Lemma 9, we know $\mathcal{J}_{\pi(\tau_x)}(f(J_x^p)^-) = \emptyset$ under S'. Meanwhile, we know at least one job with priority higher than or equal to $\pi(\tau_x)$ is pending at any time point in $[t, f'(J_j^k))$ under S'. Therefore, it must satisfy that $f(J_x^p) \geq f'(J_j^k)$, and thus $R'(J_j^k) \leq R(J_x^p)$.

At last we prove that $\pi(\tau_x) > \pi(\tau_{x+1})$. Recall that τ_{x+1} is the successor of τ_x in $\mathcal{C}$. If $\tau_{x+1} = \tau_j$, then $\pi(\tau_x) > \pi(\tau_{x+1})$ obviously holds since $\pi(\tau_x) > \pi(\tau_j)$. If $\tau_{x+1} \neq \tau_j$, we prove $\pi(\tau_x) > \pi(\tau_{x+1})$ by contradiction. Assume that $\pi(\tau_x) < \pi(\tau_{x+1})$ holds. Then we know any job J_{x+1}^q released by τ_{x+1} under S' will be assigned an effective priority lower than $\pi(\tau_x)$. Then for any job released by tasks from τ_{x+1} to π_j, its effective priority is either lower than $\pi(\tau_x)$ or equal to the priority of the task releasing it. According to Lemma 5, we know $j > x$. Reaching a contradiction with that $\pi(\tau_x) = \pi'(J_j^k)$. Therefore, we know $\pi(\tau_x) > \pi(\tau_{x+1})$.

In summary, the lemma is proved. □

An example. Lemma 10 and 11 are illustrated with Figure 3. For $\pi'(J_j^k) = \pi(\tau_j)$, take J_1^1 with $\pi'(J_1^1) = \pi(\tau_1) = 3$ as an example. Under S', J_1^1 executes in a level-3 busy period, which has the same end time under S and S'. Since J_1^1 must finish before the end of the level-3 busy period under S', the finish time of J_1^1 under S' does not exceed the end of level-3 busy period under S, which is equal to the finish time of J_1^1 under S. Then the response time of J_1^1 under S' must not exceed its largest response time under S, i.e., $R'(J_1^1) \leq \mathcal{R}(\tau_1)$.

For $\pi'(J_j^k) > \pi(\tau_j)$, take J_3^1 with $\pi'(J_3^1) > \pi(\tau_3)$ as an example. J_3^1 executes in a level-3 busy period and must finish before the end of this busy period. Further, the level-3 busy period starts with $r'(J_1^1)$ with $\pi'(J_1^1) = \pi(\tau_1) = 3$. By the case with $\pi'(J_j^k) = \pi(\tau_j)$, the end time of the level-3 busy period of J_1^1 under S' has the same end time as that under S, and J_1^1 finishes at the end time of the level-3 busy period under S. So $f'(J_3^1) \leq f(J_1^1)$. Since $r'(J_3^1) \geq r'(J_1^1)$ and $f'(J_3^1) \leq f(J_1^1)$, the response time of J_3^1 under S' must not exceed $\mathcal{R}(\tau_1)$ under S, i.e., $R'(J_3^1) \leq \mathcal{R}(\tau_1)$.

Theorem 3 *The reaction time and data age of a cause-effect chain $\mathcal{C}$ are upper-bounded by*

$$\begin{aligned} RT(\mathcal{C}) &\leq T_1^{max} + \sum_{j=2}^{|\mathcal{C}|} T_j^{max} + \chi \\ DA(\mathcal{C}) &\leq \sum_{j=1}^{|\mathcal{C}|-1} T_j^{max} + \chi \end{aligned} \tag{5}$$

where $\chi = max\left(\mathcal{R}(\tau_{|\mathcal{C}|}), \max_{\tau_w \in \mathcal{C}} \mathcal{R}(\tau_w)\right)$, and τ_w is an arbitrary task in $\mathcal{C}$ satisfying $\pi(\tau_w) > \pi(\tau_{w+1})$.

Proof: The theorem is proved by combining Theorem 1, Theorem 2, Lemma 10 and Lemma 11. □

As can be observed from Theorem 3, the results in (5) strictly dominate the results in [7] (Theorem 5.4 and 5.10).

D. Discussion

In general, DPI can be applied on each chain independently in a ECU as long as all tasks in the ECU are schedulable under original FPP scheduler to optimize the end-to-end latency, regardless of either the priority assignment or run-time schedulability, allowing it easy to be generally implemented.

Moreover, DPI can be implemented independently to chains in each individual ECU. When the input of a chain in a ECU is the output of another chain in a different ECU, i.e., chains in different ECUs are connected together, our proposed protocol and analysis techniques can be applied to each chain independently for deriving the local results on each individual ECU. Then, the holistic end-to-end delay of a chain across multiple ECUs can be integrated with local results, depending on the abstraction of the inter-connection. For instance, the inter-connection can be modeled as an independent communicating task, and the communication delay can be simply calculated considering the worst-case scenario in the sense of data propagation and included into the holistic end-to-end delay, e.g, via the Cutting Theorem in [4].

V. Implementing Determinism with DPI

The design philosophy of DPI is to optimize the end-to-end latency by leveraging a read-after-write property between consecutive tasks in a chain. This property lays a good foundation for us to develop a deterministic protocol for cause-effect chains. In the following, we propose DPI-B, which is a

combination of the proposed DPI and a Buffer Manipulation Protocol. With DPI-B, determinism is implemented with a smaller cost in terms of end-to-end latency, compared with other existing approaches to implement determinism.

A. Existing Approaches

Determinism is critical for model-based system design, which allows abstraction from implementation concerns and makes designs platform-independent. Two types of semantics of determinism are generally considered [13]: timing determinism, where the system output is always generated at fixed time instants, and data-flow determinism (also called 'semantics preserving' in [14]), where the system output is always generated based on the same input. In this paper, we consider the data-flow determinism. Formally, the definition is given by:

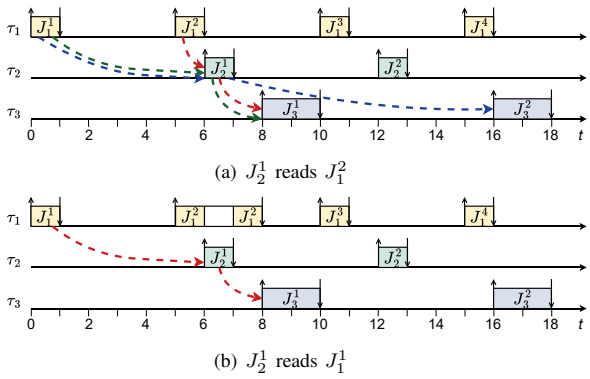
Definition 8 (data-flow determinism) *Let S be an execution sequence of all jobs with a specific release pattern on the ECU resulted by a scheduler, and S' be another execution sequence of all jobs with the same release pattern resulted by the same scheduler. Let J_{j-1}^p and J_j^q be two arbitrary jobs released by τ_{j-1} and τ_j in chain $\mathcal{C}$, and J_j^q reads the data produced by J_{j-1}^p (denoted by $J_{j-1}^p \rightarrow J_j^q$) in S. Then, we say the $\mathcal{C}$ is deterministic under the scheduler if and only if $J_{j-1}^p \rightarrow J_j^q$ holds for all S'.*

For instance, a cause-effect chain scheduled by a fixed priority preemptive scheduler with implicit communication is nondeterministic. In Figure 4, $\pi(\tau_1) = 3$, $\pi(\tau_2) = 1$ and $\pi(\tau_3) = 2$. The red dashed arrows denote the data a job reads. When the actual execution time of J_1^2 is equal to 1 (Figure 4-(a)), J_2^1 reads the data produced by J_1^2. When the actual execution time of J_1^2 is equal to 2 (Figure 4-(b)), J_2^1 reads the data produced by J_1^1. Consequently, the chain is nondeterministic.

Unfortunately, achieving determinism is not a free lunch, and usually costs a significant performance degradation in the sense of end-to-end latency. Two of the most widely used approaches to implement determinism are LET (Logical Execution Time) [8], [9] and DBP (Dynamic Buffer Protocol) [14]. Determinism in LET is implemented by fixing the time instants of the read and write operation of a job, i.e., each job reads inputs and writes outputs at its release time and absolute deadline. So the data read by a job is only decided by release patterns and independent of actual execution sequence. Determinism in DBP is implemented by determining the write-read precedence relation at the release time of jobs via manipulation of buffers, and thus independent of the actual execution sequence. The protocol works as follows:

- High-to-low: A lower-priority reader job reads the data written by the latest higher-priority writer job (a job released before or at the same time with the reader job under consideration).
- Low-to-high: A higher-priority reader job reads the data written by the predecessor job of the latest lower-priority writer job.

Both LET and DBP suffer unsatisfactory real-time performance: the end-to-end latency is lengthened to trade for determinism. In Figure 4-(a), the write-read precedence relations under DBP and LET are plotted with green and blue dashed arrows. With LET, the time to propagate a data is postponed by assuming each job finishes at its deadline. With DBP, the postponed propagation mainly comes from the low-to-high case: since a higher-priority reader job may preempt the execution of a lower-priority writer job, to guarantee determinism a reader job can only be assigned to read the data produced by the predecessor of the latest writer job.



(a) J_2^1 reads J_1^2

(b) J_2^1 reads J_1^1

Fig. 4. Illustration of determinism.

B. New Approach: DPI-B

As proved in Lemma 2 and 3, under DPI a reader job J_j^q always starts after the latest released writer job J_{j-1}^p, so the data produced by J_{j-1}^p is always ready to be read when J_j^q starts. However, DPI itself can not make the cause-effect chain deterministic, since the data a reader job reads may vary with different execution sequences: a reader job may read the data from either the latest writer job released before it or writer jobs released after it.

Next we propose a buffer manipulation protocol, which is combined with DPI to generate deterministic outputs.

The proposed buffer manipulation protocol works by determining the write-read precedence relation among jobs based on their release times, and the principle is to guarantee a reader job always reads the data produced by the latest writer job released before it, which has been written into the input buffer before the reader job starts under DPI. The details of the protocol are shown in Algorithm 1. For a pair of consecutive tasks τ_{j-1} and τ_j, a sequence of buffers $B_{j-1,j} = \{B_{j-1,j}[1], B_{j-1,j}[2], ..., B_{j-1,j}[x], ..., B_{j-1,j}[|B_{j-1,j}|]\}$ is maintained, where $B_{j-1,j}[x]$ is the x^{th} buffer, and the size of $B_{j-1,j}$, denoted by $|B_{j-1,j}|$, is calculated in Lemma 12[2]. Each buffer $B[x]$ is characterized by $\{B[x].cur, B[x].rd, B[x].num\}$, which is checked for update at each time instant. $B[x].cur$ denotes whether the data in the buffer $B[x]$ is written by the latest released writer

[2]For ease of presentation, we omit the subscripts $j-1, j$ in the following content.

job. $B[x].rd$ denotes whether the buffer $B[x]$ is assigned to at least one unfinished reader job. $B[x].num$ denotes the number of unfinished reader jobs to read $B[x]$. Let LR denote whether the latest released job is generated by the writer task τ_{j-1} ($LR = 1$) or the reader task τ_j ($LR = 0$). By Algorithm 1, at one time instant there is only one buffer $B[X]$ with $B[X].cur = 1$, and a released reader job always reads the buffer $B[X]$ with $B[X].cur = 1$, i.e., the data produced by the last writer job released before it (line 16-19). When a buffer $B[X]$ is allocated to be read by an unfinished reader job, $B[X].rd = 1$. Since a writer job only writes to the buffer with $B[X].rd = 0$, the buffer allocated to a reader job is kept for it until all jobs reading it finish ($B[X].num = 0$) and $B[X].rd$ is set to 0 (line 24). Note that when a writer job and a reader job are released at the same time, the operations of a writer job (line 3-13) are prioritized. We only consider the 'valid' readers which are released when at least one data is written into the buffer.

Algorithm 1: Buffer Manipulation Protocol

```
1  for each buffer B[x] in B do
2      B[x].cur = B[x].rd = B[x].num = 0;
3  if a writer job is released by τ_{j-1} then
4      if LR is equal to 0 then
5          Find the buffer B[X] with B[X].cur = 1;
6          Set B[X].cur = 0;
7          Find a buffer B[Y] with B[Y].rd = 0;
8          Set B[Y].cur = 1;
9          Write to the buffer B[Y];
10         Set LR = 1;
11     if LR is equal to 1 then
12         Find the buffer B[X] with B[X].cur = 1;
13         Write to the buffer B[X];
14 if a reader job is released by τ_j then
15     Set LR = 0;
16     Find the buffer B[X] with B[X].cur = 1;
17     Set the buffer with B[X].rd = 1;
18     B[X].num = B[X].num + 1;
19     Read from the buffer B[X];
20 if a reader job released by τ_j finishes then
21     Find the buffer B[X] with B[X].rd = 1;
22     B[X].num = B[X].num − 1;
23     if B[X].num is equal to 0 then
24         Set B[X].rd = 0
```

By Algorithm 1, only the data in buffer $B[X]$ with $B[X].rd = 1$ is to be read by some reader job and then propagated to downstream jobs in the chain. Next we calculate $|B_{j-1,j}|$, i.e., the minimum buffer size required by B to guarantee that no data in it is overwritten before the corresponding reader job finishes.

Lemma 12

$$|B_{j-1,j}| = \lceil \frac{\chi}{T_j^{min}} \rceil + 1$$

where χ is defined in Theorem 3.

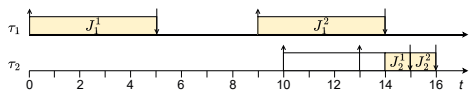


Fig. 5. The execution sequence illustrating the buffer manipulation protocol in Algorithm 1.

Time instants	Status of $B[1]$		
	$B[1].cur$	$B[1].rd$	$B[1].num$
0	1	0	0
9	1	0	0
10	1	1	1
13	1	1	2
15	1	1	1
16	1	0	0

TABLE I
ILLUSTRATION OF THE BUFFER MANIPULATION PROTOCOL IN ALGORITHM 1.

Proof: Let $|B_{j-1,j}|(t)$ be the total number of buffers of the following two types at an arbitrary time instant t: (1) buffers that have been allocated to be read by an unfinished job (2) buffers to be allocated to a forthcoming reader job. Then $|B_{j-1,j}| = \max_t |B_{j-1,j}|(t)$. We distinguish two cases.

- There are no unfinished reader jobs at t. Then by line 23-24 in Algorithm 1, there is no buffer $B[x]$ with $B[x].rd = 1$. Besides, at most one buffer is written by some writer job and to be allocated to the forthcoming reader job released after t. So $|B_{j-1,j}|(t) \leq 1$.
- There is at least one unfinished reader job at t. Let t_1, t_2 be the release time of the first and last one among these reader jobs unfinished at t, respectively. By Lemma 10 and 11, the response time of an arbitrary job does not exceed χ. Then the length of time interval $[t_1, t]$ does not exceed χ, and at most $\lceil \frac{\chi}{T_j^{min}} \rceil$ reader jobs are released in this time interval. Since one buffer is allocated to a reader job upon release, at most $\lceil \frac{\chi}{T_j^{min}} \rceil$ buffers are allocated to reader jobs, and for each of buffers $B[X]$, $B[X].rd = 1$. For writer jobs released between in $[t_2, t]$, only one buffer is allocated to them for the forthcoming reader job. So $|B_{j-1,j}|(t) \leq \lceil \frac{\chi}{T_j^{min}} \rceil + 1$.

Combining the two cases, the lemma is proved. □

The combination of DPI and the Buffer Manipulation Protocol is called DPI-B, and an example is shown in Figure 5 and Table I. $\mathcal{C} = \{\tau_1, \tau_2\}$. $\pi(\tau_1) = 2$ and $\pi(\tau_2) = 1$. During the execution, only one buffer is needed, denoted by $B[1]$. Status of $B[1]$ at critical instants are shown in Table I.

Next we prove the determinism implemented by the combination of DPI and the buffer manipulation protocol.

Theorem 4 *A cause-effect chain is deterministic with DPI-B.*

Proof: Let J_{j-1}^p and J_j^q be two arbitrary jobs released by consecutive tasks τ_{j-1} and τ_j, and $J_{j-1}^p \rightarrow J_j^q$ in one execution sequence S. By Algorithm 1, a reader job only reads the data produced by a writer job released before it, so $r(J_j^q) \geq r(J_{j-1}^p)$.

In the next, we prove the theorem by contradiction. Assume that J_j^q does not read J_{j-1}^p in an execution sequence S' of jobs with the same release pattern as S. We distinguish two cases:

- J_j^q starts before J_{j-1}^p finishes. This is impossible under DPI by Lemma 3 and 4.
- J_j^q starts after J_{j-1}^p finishes. Then the only reason for J_j^q not to read J_{j-1}^p is that the data written by J_{j-1}^p is overwritten before J_j^q reads it. Then there must exist at least one job J_{j-1}^{p+1} satisfying J_{j-1}^{p+1} finishes before J_j^q starts.
  - $r(J_{j-1}^p) \leq r(J_j^q) \leq r(J_{j-1}^{p+1})$. By the buffer manipulation protocol, J_{j-1}^{p+1} never overwrites the data in $B[x]$ written by J_{j-1}^p until J_j^q finishes and sets $B[x].rd = 0$. So it is impossible that J_{j-1}^{p+1} overwrites the data written by J_{j-1}^p, leading to contradiction.
  - $r(J_j^q) \geq r(J_{j-1}^{p+1}) \geq r(J_{j-1}^p)$. Then J_{j-1}^{p+1} must always finish before J_j^q starts and overwrites the data written by J_{j-1}^p in any arbitrary schedule S', leading to contradiction.

Combining the two cases proves the theorem. □

Comparison with LET and DBP. DPI-B outperforms both LET and DBP in terms of end-to-end latency since a job always reads the data produced by the last writer job released before it. Besides, the buffer manipulation protocol in Algorithm 1 is applicable for both schedulable and unschedulable systems, while DBP is only applicable for schedulable systems.

VI. Evaluation

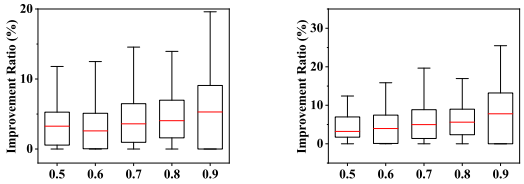
In this section, we conduct experiments with both automotive benchmark and randomly generated task sets to evaluate the proposed techniques.

A. Automotive Benchmark

Task sets are generated according to the automotive benchmarks presented in [15].

The period of each task is generated from a set $\{1, 2, 5, 10, 20, 50, 100, 200\}$, which is a subset of periods in [15][3]. The average execution time of tasks are generated with Weibull distribution, and for each generated value it is guaranteed to be located in the range of (Min, Max) of ACET (based on Table IV in [15]). Then WCET factor is generated with uniform distribution within (f_{min}, f_{max}) (based on Table V in [15]). The WCET is derived by multiplying ACET with WCET factor. We generate task sets with utilization $\{0.5, 0.6, 0.7, 0.8, 0.9\}$ (1% error is allowed). The release offset among tasks is set to 0. The utilization of each task τ_j is calculated by e_j/T_j^{min}, and the utilization of the task set is the summation of all tasks' utilization. For each task set, we first generate an initial task set with 3000 tasks, and use subset-sum approximation algorithm to derive a subset from them, and the total utilization of these subset tasks equals different target utilization [16]. Based on these tasks satisfying utilization requirement, we generate cause-effect chains. We generate the number of different periods (the so-called activation pattern in [15]) and the number of tasks for each period based on the distribution parameters based on Table VI and VII in [15]. Each chain includes 2-15 tasks, and we generate 3-6 independent chains for each task set. For each value on x-axis in Figure 6-(a) and (b), 300 task sets are generated.

[3]Since the sum of percentage from period 1 to 200 is 0.81, all share values in Table III in [15] are divided by 0.81 in the generation process.



(a) x-axis: total utilization (b) x-axis: total utilization

Fig. 6. The results with benchmark.

We randomly choose one chain from the task set and compare the results calculated by (5) and the results in [7], respectively. The improvement ratio is calculated by $\frac{bound_{Durr} - bound_{ours}}{bound_{Durr}}$. Figure 6-(a) and (b) show the results of reaction time and data age under each utilization.

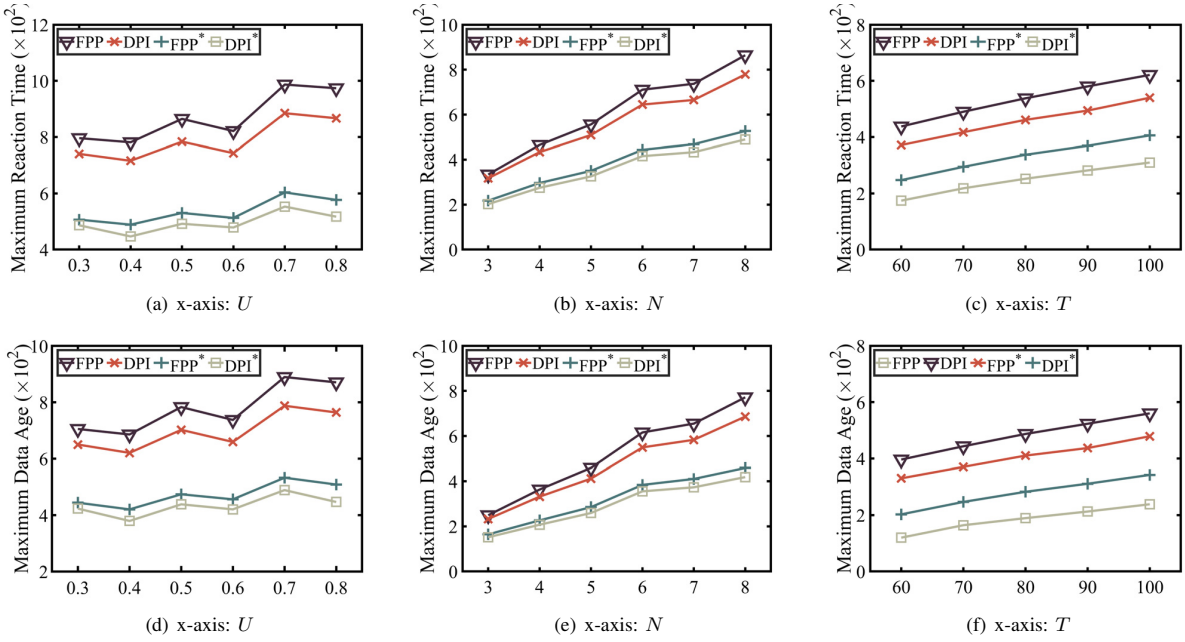
B. UUnifast Task Set Setup

The evaluation in this section includes two parts. The first part evaluates the end-to-end latency derived under DPI and fixed-priority preemptive scheduling, and the second part evaluates the end-to-end latency improvement of DPI-B over existing approaches to implement determinism, i.e., DBP and LET.

1) Comparison with FPP: We compare the reaction time and data age derived by the following approaches:

- DPI: the reaction time and data age under DPI calculated by (5).
- DPI*: the reaction time and data age observed by simulating the actual execution sequence under DPI.
- FPP: the reaction time and data age under fixed priority preemptive (FPP) scheduling calculated by [7].
- FPP*: the reaction time and data age observed by simulating the actual execution sequence under FPP, where the release pattern and actual execution time are the same as in DPI*.

Task Set Generation. We set a basic configuration: $U = 0.6$, $T = 100$ and $N = 8$, where U is the total utilization of the task set, the minimum inter-release time T_j^{min} of task τ_j is randomly generated in $[1, T]$, and N is the number of tasks in the analyzed chain. In each group of experiments we vary one parameter while keeping others unchanged. We randomly generate 15 tasks in a task set, and randomly pick some of tasks to form a chain. The utilization of each task is generated based on UUnifast Algorithm [17]. The WCET of each task τ_j is calculated by multiplying its utilization with T_j^{min}. To determine T_j^{max}, we randomly use value λ_j from $[1, 2]$ for each task and the maximum inter-release time T_j^{max} is decided by $\lambda_j \cdot T_j^{min}$. The inter-release time between consecutive jobs



(a) x-axis: U (b) x-axis: N (c) x-axis: T

(d) x-axis: U (e) x-axis: N (f) x-axis: T

Fig. 7. Comparison with FPP.

in simulation are randomly chosen in $[T_j^{min}, T_j^{max}]$. Each job executes to its WCET in simulation, and the simulation lasts until $2 \cdot LCM$, where LCM is the least common multiplier of the T_j^{max} of all tasks in the chain. The priority of tasks is assigned randomly and the schedulability is checked before further analysis. For each value on x-axis in Figure 7, 200 task sets are generated.

Results. Figure 7-(a) to (c) show the comparison of different approaches with regards to maximum reaction time. The maximum reaction time under DPI derived by both (5) and simulation is smaller than that under FPP. It is observed in Figure 7-(a) that the improvement of DPI over FPP is more obvious with increasing U. The reason is that when U grows larger, tasks tend to have larger response time bound. By comparing (5) and the results in [7], the response time bound of some tasks is no longer counted in for calculating end-to-end latency, and thus leading to smaller results. The improvement also increases with N and T, as shown in Figure 7-(b) and (c). Figure 7-(d) to (f) show the comparison of different approaches with regards to maximum data age, and similar trends can be observed.

2) Comparison with DBP & LET: We compare the reaction time and data age derived by the following approaches:

- DPI-B: the reaction time and data age observed by simulating the execution sequence under DPI and applying the proposed buffer manipulation protocol.
- DBP: the reaction time and data age observed by simulating the execution sequence under FPP and applying DBP, where the release pattern and actual execution time are the same as in DPI-B.
- LET: the reaction time and data age observed by simulating the execution sequence under FPP and applying LET, where the release pattern and actual execution time are the same as in DPI-B.

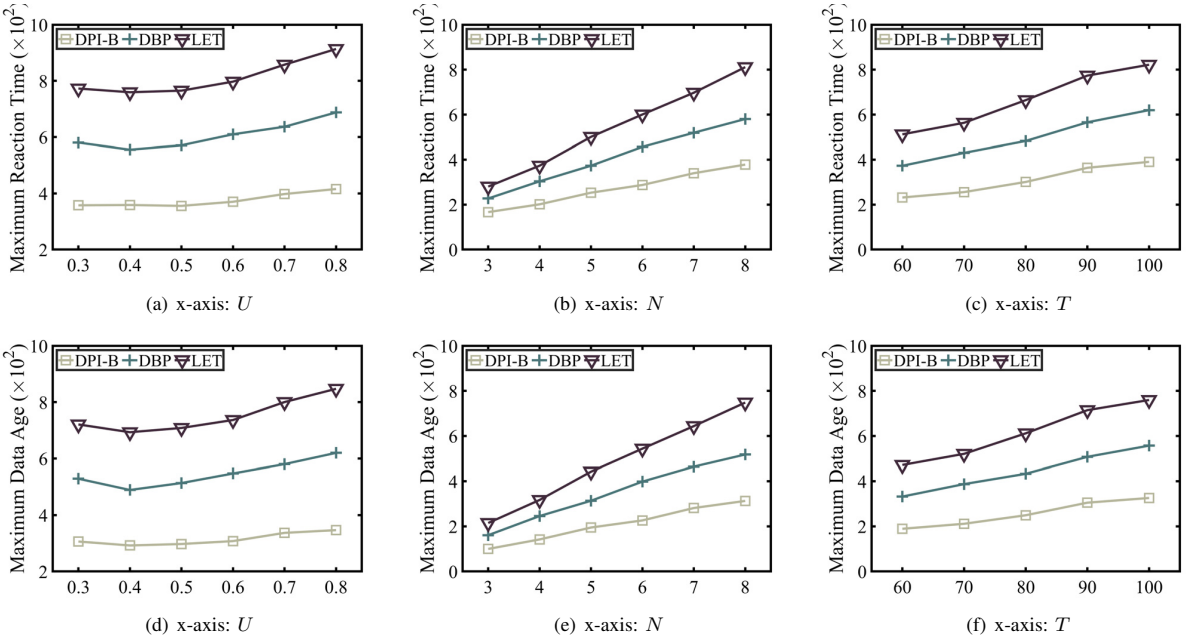
Task Set Generation. The generation of task sets is the same as in last section (Section VI.B.1) except the difference below: We consider periodic tasks with offsets, so we no longer generate T^{max}. The offset is randomly chosen in $[1, T_j^{min}]$. Each job executes to its WCET in simulation, and the simulation lasts until $O_{max} + 2 \cdot LCM$ [18], where LCM is the least common multiplier of the T_j^{min} of all tasks in the chain, and O_{max} is the maximum offset among all tasks. For each value on x-axis in Figure 8, 200 task sets are generated.

Results. Figure 8-(a) to (c) show the comparison of different approaches with regards to maximum reaction time. DPI-B generates the smallest end-to-end latency since a reader job always reads the data produced by the last released writer job. The performance of LET indicates that it implements determinism with heavy cost in terms of end-to-end latency. Figure 8-(d) to (f) show the comparison of different approaches with regards to maximum data age, and similar trends can be observed.

VII. Related Work

In complex real-time systems, two types of tasks chains are generally considered [6]: *trigger chains* [19]–[21] where a predecessor task triggers the release of a successor task, and cause-effect chains, where tasks are individually triggered and hence over- or under- sampling may occur. This paper considers cause-effect chains.

End-to-end latency of cause-effect chains in multi-rate systems is important in many safety-critical domains, such as



(a) x-axis: U (b) x-axis: N (c) x-axis: T

(d) x-axis: U (e) x-axis: N (f) x-axis: T

Fig. 8. Comparison with DBP & LET.

automotive and avionics [1] [2]. [11] proposed a framework for calculating end-to-end latency in multi-rate systems, and defined different types of semantics of end-to-end latency. Since then, a large proportion of work has been conducted focusing on periodic tasks under implicit communication. [3], [22]–[24] calculated the reaction time and data age by summing up the inter-release time of communicating tasks. [4] derived upper bound of reaction time and data age for cause-effect chains composed of periodic tasks in asynchronized systems by utilizing the cutting theorem. [5], [6], [25] calculated data age by synthesizing a partial ordering on the task's jobs via read- and data- intervals, and [26] present an approach to extend the Real-Time Systems Compiler to enforce the job-level dependencies in actual multicore schedules. Specially, [5] considered the EDF scheduling, which is similar to proposed DPI in our work. However, the analysis [5] is based on periodic tasks and can not be applied for sporadic tasks. For sporadic tasks, [27] proposed end-to-end latency analysis and period optimization techniques to guarantee the end-to-end timing constraints are satisfied (applicable for both periodic and sporadic tasks). [7] proposed immediate forward and backward job chains, and calculated tighter end-to-end latency for sporadic tasks compared with [27]. This paper adopted the definitions in [7] and proposed dynamic priority inheritance protocol to optimize end-to-end latency, which is not considered in [7].

LET (Logical Execution Time) [8], [9] and DBP (Dynamic Buffering Protocol) [14] have been proposed to implement determinism in multi-rate real-time systems, and the fundamental ideas differ. LET semantics fixes the length of time interval from tasks reading input and writing outputs, thus called logical execution time. Consequently, the read and write operations happen at deterministic time instants and independent of actual execution sequence. [28] calculated the age latency and proposed heuristic to optimize the jitter of data age by adjusting release offsets of tasks. [29] developed mathematical and algorithmic tools to analyze graphs and computed the data age of the system. DBP puts no limitation on when a job writes or reads. Rather, by manipulating pointers and buffers, it defines the partial order between writer and reader jobs. [30] computed the reaction time of synchronous periodic tasks. The proposed buffer manipulation protocol in this work shares the similar idea of DBP: deciding the write-read precedence relation upon release time. However, the proposed buffer manipulation protocol is applicable for both schedulable and unschedulable systems, while DBP is only applicable for schedulable systems. Besides implementing determinism, benefiting from the priority adjustment by DPI, the end-to-end latency is reduced compared with [14], by optimizing data propagation interval from a lower-priority writer task to higher-priority reader task.

Priority assignment has drawn much attention due to its critical impacts on execution sequence. [31] summarized a series of work exploring the priority assignment strategy under fixed-priority scheduling. Another work considering priority assignment for satisfying end-to-end timing constraints is [32], which intends to minimize the average WCRT of selected tasks and messages. [33] proposed a priority assignment heuristic for optimizing the end-to-end latency for cause-effect chains communicating with DBP, and also proposed that finding the optimal priority assignment for cause-effect chains with implicit communication paradigm is difficult due to its heavy

dependency on actual execution sequence.

VIII. Conclusion

In this paper, a dynamic priority inheritance protocol (DPI) is proposed to optimize the maximum reaction time and maximum data age in cause-effect chains. Moreover, we propose a buffer manipulation protocol, which combined with DPI implements data-flow determinism and optimizes the end-to-end latency of cause-effect chains compared with other approaches to implement data-flow determinism.

IX. Acknowledgement

This work was partially supported by the National Natural Science Foundation of China (NSFC 62102072), the Research Council of Hong Kong GRF under Grants 11208522 and 15206221, and National Frontiers Science Center for Industrial Intelligence and Systems Optimization, Shenyang 110819, China.

References

[1] A. Hamann, D. Dasari, S. Kramer, M. Pressler, and F. Wurst, "Communication centric design in complex automotive embedded systems," in *ECRTS*, 2017.

[2] J. Forget, F. Boniol, and C. Pagetti, "Verifying end-to-end real-time constraints on multi-periodic models," in *ETFA*, 2017.

[3] T. Kloda, A. Bertout, and Y. Sorel, "Latency analysis for data chains of real-time periodic tasks." in *ETFA*, 2018.

[4] M. Günzel, K.-H. Chen, N. Ueter, G. v. d. Brüggen, M. Dürr, and J.-J. a. Chen, "Timing analysis of asynchronized distributed cause-effect chains." in *RTAS*, 2021.

[5] T. Klaus, M. Becker, S.-P. Wolfgang, and P. Ulbrich, "Constrained data-age with job-level dependencies: How to reconcile tight bounds and overheads," in *RTAS*, 2021.

[6] M. Becker, D. Dasari, S. Mubeen, M. Behnam, and T. Nolte, "End-to-end timing analysis of cause-effect chains in automotive embedded systems," *JSA*, 2017.

[7] M. Durr, G. V. D. Bruggen, K. Chen, and J. Chen, "End-to-end timing analysis of sporadic cause-effect chains in distributed systems," *TECS*, 2019.

[8] C. M. Kirsch and A. Sokolova, "The logical execution time paradigm," in *Springer Berlin Heidelberg, Berlin, Heidelberg*, 2012.

[9] T. Henzinger, B. Horowitz, and C. Kirsch, "Giotto: A time-triggered language for embedded programming," *IEEE*, 2003.

[10] J. P. Lehoczky, L. Sha, and Y. Ding, "The rate monotonic scheduling algorithm: Exact characterization and average case behavior," in *RTSS*, 1989.

[11] N. Feiertag, K. Richter, J. Nordlander, and J. Jonsson, "A compositional framework for end-to-end path delay calculation of automotive systems under different path semantics." *Workshop on Compositional Theory and Technology for Real-Time Embedded Systems*, 2009.

[12] J. Lehoczky, "Fixed priority scheduling of periodic task sets with arbitrary deadlines," in *RTSS*, 1990.

[13] K. Gemlau, L. Kohler, R. Ernst, and S. Quinton, "System-level logical execution time: Augmenting the logical execution time paradigm for distributed real-time automotive software," *ACM Trans. Cyber-Phys. Syst.*, 2021.

[14] P. Caspi, N. Scaife, C. Sofronis, and S. Tripakis, "Semantics-preserving multitask implementation of synchronous programs," *TECS*, 2008.

[15] S. Kramer, D. Ziegenbein, and A. Hamann, "Real world automotive benchmark for free," in *WATERS*, 2015.

[16] G. v. d. Brüggen, N. Ueter, J. Chen, and M. Freier, "Parametric utilization bounds for implicit-deadline periodic tasks in automotive systems." in *RTNS*, 2017.

[17] E. Bini and G. C. Buttazzo, "Measuring the performance of schedulability tests," *RTS*, 2005.

[18] J. Y.-T. Leung and J. Whitehead, "On the complexity of fixed-priority scheduling of periodic, real-time tasks," *Performance evaluation*, 1982.

[19] J. Schlatow and R. Ernst, "Response-time analysis for task chains in communicating threads," in *RTAS*, 2016.

[20] ——, "Response-time analysis for task chains with complex precedence and blocking relations," *TECS*, 2017.

[21] S. Schliecker and R. Ernst, "A recursive approach to end-to-end path latency computation in heterogeneous multiprocessor systems," in *CODES+ISSS*, 2009.

[22] T. Kloda, A. Bertout, and Y. Sorel, "Latency upper bound for data chains of real-time periodic tasks," in *JSA*, 2020.

[23] J. Ren, X. He, J. Zhou, H. Ge, G. Wu, and G. Tan, "Efficient latency bound analysis for data chains of real-time tasks in multiprocessor systems," in *DATE*, 2020.

[24] R. Bi, X. Liu, J. Ren, P. Wang, H. Lv, and G. Tan, "Efficient maximum data age analysis for cause-effect chains in automotive systems," in *DAC*, 2022.

[25] M. Becker, D. Dasari, S. Mubeen, M. Behnam, and T. Nolte, "Synthesizing job-level dependencies for automotive multi-rate effect chains," in *RTCSA*, 2016.

[26] T. Klaus, F. Franzmann, M. Becker, and P. Ulbrich, "Data propagation delay constraints in multi-rate systems – deadlines vs. job-level dependencies," in *RTNS*, 2018.

[27] A. Davare, Q. Zhu, M. D. Natale, C. Pinello, S. Kanajan, and A. L. Sangiovanni-Vincentelli, "Period optimization for hard real-time distributed automotive systems." in *DAC*, 2007.

[28] J. Martinez, I. Sañudo, and M. Bertogna, "Analytical characterization of end-to-end communication delays with logical execution time," *TCAD*, 2018.

[29] A. Kordon and N. Tang, "Evaluation of the age latency of a real-time communicating system using the let paradigm," in *ECRTS*, 2020.

[30] J. Abdullah, G. Dai, and W. Yi, "Worst-case cause-effect reaction latency in systems with non-blocking communication," in *DATE*, 2019.

[31] R. I.Davis, L. Cucu-Grosjean, M. Bertogna, and A. Burns, "A review of priority assignment in real-time systems," *JSA*, 2016.

[32] Y. Zhao, V. Gala, and H. Zeng, "A unified framework for period and priority optimization in distributed hard real-time systems," *TCAD*, 2018.

[33] Y. Tang, X. Jiang, N. Guan, D. Ji, X. Luo, and W. Yi, "Comparing communication paradigms in cause-effect chains," *TC*, 2022.